Scalable Behaviour Cloning on Browser Using via Skill Distillation

Internet users collectively perform an enormous range of skilled work through web browsers, from software development and document editing to search, forms, and enterprise workflows. This activity is a highly scalable but under-exploited source of reusable browser skills. We study scalable behavior cloning for browser agents via distributed skill summarization, converting user interaction trajectories into compact natural-language skills that agents can read, retrieve, reuse, and compose directly. We show that simple trajectory summarization alone can improve benchmark performance through behavior cloning, and further introduce a distributed framework for scalable skill collection and reuse across users and tasks. This suggests that the scalability of browser agents may come less from manually designed tasks and more from the collective skills already expressed by internet users. Our project is available at: xxx.

Date: June 22, 2026

Einsia

1 Introduction

Web browsers have become the universal execution interface for modern digital work. Developers move between GitHub, continuous integration systems, and documentation; operators update dashboards and enterprise workflows; researchers search, organize, and transfer information across online tools; and everyday users complete tasks in email, calendars, shopping sites, forms, and software-as-a-service applications. A browser agent that can reliably use this interface would therefore provide a general execution layer for digital labor. Early work such as WebShop showed how language agents could act in grounded web environments (Yao et al., 2023), while Mind2Web broadened the problem toward generalist web agents over real websites (Deng et al., 2023). More recent benchmarks, including WebArena and VisualWebArena, make the challenge increasingly realistic by requiring long-horizon interaction, multimodal observation, element grounding, and task-level evaluation (Zhou et al., 2024; Koh et al., 2024). WorkArena and BrowserGym further extend this line toward common knowledge-work tasks and standardized research infrastructure (Drouin et al., 2024; Chezelles et al., 2025).

At the same time, browser agents have access to a potentially scalable source of supervision: interaction traces produced by humans and agents during real web tasks. These traces encode more than clicks. Mind2Web demonstrates that demonstrations over real websites can supervise action grounding and next-step prediction (Deng et al., 2023); WebLINX adds multi-turn, dialogue-conditioned navigation traces collected from real website use (Lù et al., 2024); and AgentTrek explores trajectory synthesis from web tutorials as another route to scalable agent data (Xu et al., 2025). Taken together, these traces contain practical strategies for decomposing work, locating relevant pages, selecting candidate elements, recovering from mistakes, and recognizing when a subtask has been completed.

However, raw browser traces are a poor unit of reusable behavioral supervision. They are long, noisy, redundant, and tightly coupled to incidental page states. Low-level action targets depend on coordinates, DOM structures, login states, page versions, and website-specific implementations, so direct cloning of coordinates, element identifiers, or next-step actions can easily learn brittle page-specific patterns rather than transferable workflow competence. WebAgent explicitly combines planning, long-context understanding, and program synthesis to handle real-world web interaction beyond flat action imitation (Gur et al., 2024). SeeAct shows that grounding is itself a central bottleneck for vision-language web agents (Zheng et al., 2024), while WebVoyager studies end-to-end multimodal web agents under realistic browsing conditions (He et al., 2024a). The same issue appears beyond browsers: OSWorld highlights how open-ended computer-use tasks require robust observation

and action abstractions rather than narrow UI-state memorization (Xie et al., 2024).

We argue that the reusable unit of browser behavior is not an individual click, nor even an entire task trajectory, but a transferable natural-language skill. A skill describes a reusable subtask strategy: its goal, applicability conditions, key observations, element-selection principles, action templates, and common failure-recovery procedures. This view builds on evidence that language agents can benefit from persistent verbal experience. Reflexion stores textual feedback to improve later decisions (Shinn et al., 2023), ExpeL extracts reusable lessons from prior trials (Zhao et al., 2024), and Voyager maintains an expanding skill library for open-ended embodied exploration (Wang et al., 2023). Recent work on inducing programmatic skills further suggests that agent behavior can be represented and reused above the level of raw action sequences (Wang et al., 2025b). In our setting, skill summarization turns noisy low-level browser traces into higher-level semantic supervision, making behavior cloning less dependent on fragile page-specific actions.

This paper proposes a scalable framework for converting browser interaction traces into reusable natural-language skills. The framework cleans and normalizes trajectories, segments them into successful subtrajectories and failure-recovery episodes, applies distributed skill summarization to extract candidate skills, and then clusters, deduplicates, filters, and indexes them into a skill library. At inference time, a browser agent retrieves and composes relevant skills, then grounds skill-level intent into semantic element selection and executable browser actions. The resulting formulation is complementary to modular agent-training frameworks such as Agent Lumos (Yin et al., 2024) and to standardized web-agent environments such as BrowserGym (Chezelles et al., 2025).

We evaluate whether this skill-centric behavior cloning paradigm improves browser-agent behavior across diverse tasks. Our evaluation considers not only next-action prediction, but also end-to-end task success, recovery from failures, skill reuse, cross-site transfer, and robustness to page perturbations.

Contributions. This paper makes the following contributions:

- We observe that skill summarization can effectively extract high-level semantic structure from low-level browser demonstrations, reducing reliance on brittle coordinate- or element-level behavior cloning, and we verify that these summaries improve agent performance.
- We propose a scalable trace-to-skill framework that converts large-scale browser interaction trajectories into a retrievable, composable, and executable natural-language skill library.
- We validate the framework across a diverse set of browser tasks, measuring next-action prediction, end-to-end task success, failure recovery, skill reuse, cross-site generalization, and robustness to page perturbations.

2 Related Work

Web and computer-use agents. Research on browser agents has progressed along three coupled fronts: environments, benchmarks, and end-to-end policies. Early work established grounded language-agent interaction in controlled settings—WebShop in a simulated storefront (Yao et al., 2023) and Mind2Web over real websites (Deng et al., 2023)—while WebArena and VisualWebArena made evaluation realistic by demanding long-horizon interaction, multimodal perception, element grounding, and programmatic success checks (Zhou et al., 2024; Koh et al., 2024); WorkArena and BrowserGym extended this toward enterprise knowledge work and a common research infrastructure (Drouin et al., 2024; Boisvert et al., 2025; Chezelles et al., 2025). A parallel line strengthens the policy itself: WebVoyager and OpenWebVoyager study multimodal agents under realistic browsing (He et al., 2024a,b), SeeAct isolates grounding as the central bottleneck for vision-language agents (Zheng et al., 2024), and WebAgent couples planning, long-context reading, and program synthesis (Gur et al., 2024). Beyond the browser, OSWorld generalizes the problem to open-ended computer use and argues for robust observation–action abstractions over narrow UI-state memorization (Xie et al., 2024), and large-scale efforts such as ScaleCUA and OpenCUA characterize how competence scales with data and supervision (Liu et al., 2026; Wang et al., 2025a). More recent systems push capability through reinforcement learning and information-seeking objectives (Li et al., 2026; Wu et al., 2025; Vattikonda et al., 2025), process reward modeling (Chae et al., 2025), structured or hierarchical exploration (Yang et al., 2025; Gandhi and Neubig,

2026; Shen et al., 2025), and tool or context abstractions (Prabhu et al., 2026; Ye et al., 2026). The common thread is that browser competence improves with scale, abstraction, and richer interaction. Our contribution is orthogonal to all three fronts: rather than proposing new environments, rewards, or training objectives, we treat the trajectories already produced by ordinary internet users as a scalable supervision signal and ask how to convert them into reusable procedural knowledge.

Learning from web demonstrations. A substantial body of work uses human or agent trajectories as supervision. Mind2Web shows that demonstrations over real websites can supervise action grounding and next-step prediction (Deng et al., 2023); WebLINX collects multi-turn, dialogue-conditioned navigation traces from real-world use (Lù et al., 2024); and AgentTrek synthesizes trajectories from web tutorials as an alternative route to scalable data (Xu et al., 2025). Such traces implicitly encode how people decompose work, locate pages, select elements, recover from mistakes, and recognize completion. The difficulty is one of representation: raw traces are long, noisy, and tightly coupled to incidental page state, so cloning coordinates, selectors, or next-step actions directly tends to capture brittle, page-specific correlations rather than transferable competence. We share the goal of exploiting demonstrations at scale but take the reusable unit of supervision to be a transferable natural-language *skill* rather than a single action or a full trajectory, and we explicitly abstract away coordinates and selectors during distillation so that the guidance survives changes in page version and authentication state.

Skills, experience, and distributed memory. Our formulation builds on evidence that language agents benefit from persistent verbal experience. Reflexion converts outcome feedback into textual self-notes that improve later attempts (Shinn et al., 2023), ExpeL extracts reusable lessons across trials (Zhao et al., 2024), and Voyager maintains an expanding library of executable skills for open-ended embodied exploration (Wang et al., 2023). Related work induces programmatic skills that represent behavior above raw action sequences (Wang et al., 2025b), while modular frameworks such as Agent Lumos decouple planning, grounding, and execution into reusable components (Yin et al., 2024). A complementary line studies how experience scales when it is distributed: federated and distributed lifelong learning share knowledge across many private, non-IID clients or agents (McMahan et al., 2017; Rostami et al., 2018); LLM memory sharing pools useful experiences across agents (Gao and Zhang, 2024); and robotics efforts such as Open X-Embodiment show that heterogeneous demonstrations across platforms can support transfer (Open X-Embodiment Collaboration et al., 2024). We differ in source and representation: we distill *human* browser trajectories from many users, websites, and tasks into an auditable skill graph, so many trajectories can support one skill, one trajectory can yield several skills, and retrieved skills can be composed for new browser tasks.

3 Method

BrowserBC is a *skill-centric* behavior cloning framework for browser agents. Rather than imitating demonstrations as flat sequences of low-level actions, BrowserBC distills user interaction trajectories into reusable, natural-language *skills*. Each skill encodes a transferable procedure for a class of browser tasks, specifying when it applies, which information must be preserved across steps, how progress is made, how completion is verified, and how common failures are diagnosed and recovered from. At inference time, the agent retrieves the skills that are semantically relevant to the current task, conditions its action generation on them, and grounds every resulting action in the live browser state.

This design shifts the unit of behavior cloning from an atomic action to a reusable skill, so that a trajectory is treated not as a script to be replayed verbatim but as *evidence* of the procedural knowledge underlying a successful interaction. The distinction is particularly consequential on the web, where the same semantic operation may be realized through markedly different layouts, DOM structures, widgets, and interaction flows: an action that is optimal on one page may be meaningless or even harmful on another. By cloning skills rather than surface actions, BrowserBC transfers browser competence across websites and tasks while substantially reducing its dependence on brittle, page-specific implementation details.

[TODO: Figure] Figure **??** summarizes the four stages of BrowserBC: behavioral evidence abstraction (§3.2), procedural skill distillation (§3.3), library consolidation (§3.4), and skill-conditioned execution (§3.5). We first formalize the skill-centric view of behavior cloning that ties these stages together.

3.1 Problem Formulation

We denote a browser trajectory as $\tau = (x, e_1, \ldots, e_T, y)$, where x is a natural-language task instruction, $e_t = (o_t, a_t, f_t)$ is the t-th event comprising the observation o_t, the user action a_t, and the execution feedback f_t (e.g., page transitions, validation messages, or completion signals), and y is the task outcome when available.

A standard behavior cloning objective fits a low-level policy $\pi(a_t \mid x, o_{\leq t})$ that maps the task and interaction history directly to the next action. In browser environments, however, low-level actions are weak carriers of transferable behavior. The same intent may map to different selectors, coordinates, or input mechanisms across pages, while visually similar actions may carry different procedural meaning depending on the task state. As a result, cloning raw actions tends to capture page-specific correlations rather than the reusable skills that explain why a demonstration succeeds.

We instead cast behavior cloning over a *skill library* $\mathcal{L} = \{s_i\}_{i=1}^{N}$, where each s_i is a natural-language procedure distilled from one or more trajectories. Given a new instruction $x^\star$ and browser history $h_t^\star = o_{\leq t}^\star$, the agent retrieves a compact, task-relevant subset of skills and predicts the next action through a *skill-conditioned* policy:

$$\mathcal{R}_t = \mathcal{R}(x^\star, h_t^\star, \mathcal{L}), \quad |\mathcal{R}_t| \ll |\mathcal{L}|, \qquad a_t \sim \pi_\theta(\cdot \mid x^\star, h_t^\star, \mathcal{R}_t). \tag{1}$$

The retrieved skills $\mathcal{R}_t$ do not replace perception or grounding; rather, they supply *procedural constraints* on what the agent should attend to, which values it must preserve, what intermediate progress looks like, when a task should terminate, and how to recover when a step fails. In this view, demonstrations supervise the agent through reusable strategies and explicit completion criteria, rather than through one-step imitation of individual actions.

3.2 Behavioral Evidence Abstraction

The first stage converts raw trajectories into structured *evidence*. Browser logs are inherently noisy and heterogeneous, often containing redundant cursor motion, accidental clicks, repeated attempts, idle waits, and private content that is irrelevant or even harmful to retain. To obtain a clean substrate for distillation, we normalize each trajectory into a common event representation, filter out interactions that do not contribute to task progress, and segment the remainder into semantically coherent sub-procedures rather than arbitrary fixed-length windows, so that each segment corresponds to a meaningful unit of behavior.

For each segment we extract an *evidence unit*

$$u = \big(x,\ c^{\text{pre}},\ b,\ c^{\text{post}},\ f,\ y\big), \tag{2}$$

consisting of the instruction x, the contexts c^{pre} and c^{post} that summarize the state immediately before and after the segment, a summary b of the demonstrated behavior, the feedback f observed during the segment, and the outcome y. The context summaries are designed to preserve task-relevant information—visible affordances, required input fields, user-provided values, applicable constraints, and success or failure signals—while discarding non-transferable artifacts such as raw coordinates, DOM selectors, transient identifiers, and personally identifying content. This abstraction *lifts* demonstrations from page-specific action traces into procedural evidence that generalizes across browser states, and it forms the basic input from which skills are subsequently distilled.

3.3 Procedural Skill Distillation

Given a set of evidence units $\mathcal{U}_s$ and optional metadata m_s (e.g., task category, website family, or source identifiers), BrowserBC distills an agent-readable skill $\hat{s} = D(\mathcal{U}_s, m_s)$. A skill is represented as a structured natural-language *card* with a fixed set of fields: intent, applicable scope, preconditions, execution steps, progress milestones, terminal evidence, failure modes, recovery strategies, negative constraints, and provenance. Together, these fields make a skill directly consumable by a language-model agent while keeping it human-auditable and easy to update. The distillation model is explicitly instructed to retain reusable procedural structure while stripping away session-specific artifacts—private text, exact coordinates, DOM paths, or one-time authentication state—unless such details are genuinely essential to the task semantics.

Crucially, a distilled skill specifies *what* should be accomplished and *how* to reason about progress toward it, not how to replay a particular demonstration. A form-filling skill, for instance, instructs the agent to identify fields by their semantic labels and to preserve task-provided values, rather than to reuse a specific coordinate or selector that happened to work in the source trajectory. Skills may be induced from a single successful trajectory—which already captures the structure of one viable solution—or consolidated from multiple attempts, in which successful runs strengthen the execution guidance while failed runs expose missing preconditions and motivate explicit recovery strategies.

3.4 Library Consolidation via Skill Graphs

A growing collection of skills is only useful if it remains coherent and non-redundant. Whenever a candidate skill $\hat{s}$ is produced, BrowserBC therefore decides whether to *add* it as a new node, *merge* it into an existing skill, or register it as a *specialization* of a more general one. Two skills are merged when they share a compatible intent, preconditions, steps, effects, and terminal evidence, and are kept separate when they apply under different conditions, require different information, or encode mutually conflicting constraints.

We organize the resulting library as a lightweight *skill graph* $\mathcal{G} = (\mathcal{L}, \mathcal{E})$, whose nodes are distilled skills and whose edges encode simple relations among them—temporal dependency, specialization, alternative procedures for a shared subgoal, or incompatibility under the same state. This structure lets a generic procedure (e.g., form filling) link to its specializations (such as payment or profile updates) and to the recovery skills that handle its characteristic failures, without collapsing them into a single flat entry. The graph improves scalability along three axes: it consolidates repeated demonstrations into reusable nodes instead of accumulating isolated examples; it restricts retrieval and updates to the relevant regions of the skill space; and it supports incremental refinement, since a new trajectory updates only the affected skills and their immediate neighbors. Importantly, the graph serves as an organizational scaffold rather than a planner: the learned behavioral unit remains the distilled natural-language skill, and edges merely structure how those skills are stored, retrieved, and revised.

3.5 Skill-Conditioned Execution

At inference time, the retrieval operator $\mathcal{R}$ in Eq. (1) selects the most relevant skills by their semantic similarity to the task and, when such information is available, their compatibility with the current browser context. We deliberately avoid introducing a specialized retrieval architecture or an explicit symbolic planner; retrieval simply serves to expose the agent to the prior knowledge most likely to be useful at the current step. The selected skills $\mathcal{R}_t$ are then inserted into the agent's context as natural-language guidance, and the agent is responsible for grounding them into executable actions against the live page.

Skills are thus neither executable scripts nor demonstrations to be replayed verbatim: they bias the agent toward distilled behavior patterns, while each concrete action is still chosen with respect to the current state. When applying a form-filling skill, for example, the agent identifies the required fields by their labels, transfers the task-provided values into them, monitors for validation messages, and stops once a success confirmation appears—with the concrete fields and layout read from the current page rather than copied from any source trajectory. This combination preserves the data efficiency of behavior cloning while avoiding the brittleness of action replay, allowing a single skill to guide behavior across different websites and interaction mechanisms.

3.6 Evidence Regimes

Finally, we note that the same formulation gracefully spans multiple *evidence regimes* that differ only in how much demonstration data is available. A single successful trajectory yields a narrow skill that captures one solution pattern; repeated attempts on the same task consolidate successes and failures into a more robust skill with clearer preconditions and recovery guidance; and large-scale interaction data accumulates a broad library of reusable skills spanning many users, websites, and task families. This unified view also clarifies why even simple trajectory summarization already constitutes a form of behavior cloning: a summarized trajectory becomes a procedural skill that conditions future action generation. The scalability of BrowserBC therefore stems precisely from this conversion of many individual demonstrations into compact, reusable, and

continually refinable skills—competence that is learned without any dependence on brittle, low-level action imitation.

4 Experiments

4.1 ClawBench

4.2 WebArena (@zbs)

We evaluate whether distilling human browser trajectories into reusable natural-language skills improves end-to-end task success for a strong browser agent on WebArena. Our central question is direct: holding the agent, environment, and evaluation fixed, does equipping the agent with BrowserBC skills raise its success rate, and on what kinds of tasks? We first describe the evaluation protocol (§4.2.1), then report the main skill-on / skill-off comparison on WebArena (§4.2.2), analyze where skills help and where they do not (§4.2.3), and finally summarize the integrity safeguards that make the comparison trustworthy (§4.2.5).

4.2.1 Setup

Benchmark. We evaluate on the hard split of WebArena (Zhou et al., 2024), comprising 258 human-verified tasks across five self-hosted sites (GitLab, an e-commerce storefront and its admin panel, a Reddit-style forum, and an OpenStreetMap-based map service). Each task is scored by its original programmatic checker, which inspects either the agent's final answer (*string-match*: exact, fuzzy, or required-substring), the resulting page URL (*url-match*), or the live page state via DOM/JavaScript locators (*program-html*). We report task success rate (SR), the fraction of tasks whose checker returns full credit. To make the analysis interpretable, we group tasks into three categories: information-seeking *read-only* tasks (85), state-changing *mutation* tasks (154), and *map* routing/geocoding tasks (19).

Agent and harness. Both conditions use the same browser agent: a Claude model driven through an agentic CLI loop, acting on the live page through a Playwright-based Model-Context-Protocol (MCP) browser that exposes both a rendered *screenshot* (vision) and a structured *DOM/accessibility snapshot* at every step, with element grounding and actions issued back over CDP. This multimodal, official-aligned harness is held identical across conditions; the only variable is whether BrowserBC skills are injected into the agent context. Each task is given the same step and wall-clock budget in both conditions.

Conditions. We compare two configurations of the identical agent: **(i) skill-off (base)**, the agent with no external skills; and **(ii) skill-on (BrowserBC)**, the same agent additionally given the distilled natural-language skill for the task family, retrieved from the human-trajectory skill library. The skills are produced by the pipeline of §3: human demonstrations are normalized, segmented, and distilled into procedural skills describing applicability conditions, a generalized execution procedure, progress milestones and terminal evidence, and common failure-recovery steps. Crucially, skills describe *how to perform* a task family at the level of page semantics and element-selection principles; they contain no benchmark answers and no checker-specific values (§4.2.5).

4.2.2 Main Result: Skills Improve Browser-Agent Success

Table 1 reports the comparison on WebArena-Hard: the identical agent is run on all 258 tasks with skills off and with the BrowserBC skill library, and we report success rate. Equipping the agent with distilled human-trajectory skills raises overall success from 60.5% to 81.4%, a +20.9 point absolute improvement (+34.5% relative) while leaving the agent, environment, evaluation, and budget unchanged. The gain is large across every task category and concentrates where procedural knowledge matters most. We report BrowserBC as the skill-augmented agent that retrieves and applies skills; a strict ablation that instead forces the agent to follow every retrieved skill verbatim—even when live page evidence contradicts it—reaches 77.5%, so part of the gain comes from the agent applying skills selectively rather than blindly, consistent with the confidence-aware design of §3.

Table 1 Task success rate (SR) on WebArena-Hard (258 tasks) for the identical browser agent with skills off (base) and with the BrowserBC skill library. Skills are distilled from human browser trajectories and contain procedural guidance only. Δ is the absolute improvement in percentage points.

Category	# Tasks	Skill-off (base)	BrowserBC	Δ
Read-only	85	56.5	85.9	+29.4
Mutation	154	62.3	78.6	+16.3
Map	19	63.2	84.2	+21.0
All	258	**60.5**	**81.4**	**+20.9**

4.2.3 Analysis

Where do skills help most? The largest improvement is on read-only, information-seeking tasks (+29.4 points). These tasks fail in the base agent not because the answer is unreachable, but because the agent searches the wrong way, stops early, or reads an incomplete slice of a long result list or table. The distilled skills encode exactly the transferable procedure a human used—which page and sort order to open, which products or rows are genuinely relevant, and how to read minimum and maximum values—turning a fragile search into a reliable one. Map tasks improve comparably (+21.0 points): the skills supply the correct routing/geocoding procedure (which service path to query and how to land on the destination's map page) that the base agent often reaches only unreliably. Mutation tasks improve substantially as well (+16.3 points): skills reliably fix *navigation and progress recognition* (reaching the correct form, knowing which terminal page constitutes completion), with the residual failures dominated by genuine multi-field execution precision, where the agent must set many interdependent fields correctly in a single form.

What remains hard. A manual audit of the residual failures shows they are concentrated in *agent execution limits* rather than missing knowledge: multi-field admin-panel forms where one of several required attributes is omitted, search aggregations where one item of a list is dropped, and tasks requiring an exact target among several plausible candidates. For these, providing an even more detailed skill does not help—the procedure is already correct, but the agent's step-by-step execution is not perfectly reliable. This delineates a clean boundary: BrowserBC closes the gap caused by *not knowing the procedure*, and the remaining headroom is *procedure-following fidelity*, which is a property of the underlying agent rather than of the skill library.

Skills are not universally beneficial. The improvement is a strong net effect, not a uniform one. On a small fraction of tasks (10 of 258, 3.9%), blindly following the retrieved skill makes the agent fail a task it solves on its own, and these regressions persist across repeated runs rather than being sampling noise. They occur when an over-specific or imperfectly-distilled skill anchors the agent to one strategy and discourages the simpler exploration that would otherwise have succeeded. This is exactly the gap between the strict always-follow ablation (77.5%) and BrowserBC (81.4%): allowing the agent to apply a skill selectively—and to defer to live page evidence when it conflicts with the skill—recovers these regressions. It exposes an honest property of behavior cloning from summaries (a skill encodes a prior, and an over-confident prior can mislead an otherwise-capable agent) and motivates the confidence-aware, evidence-weighted retrieval of §3, so that the agent trusts a retrieved skill in proportion to its demonstrated reliability rather than following it unconditionally.

4.2.4 Failure Case Study

To make the residual headroom concrete, we group the surviving failures of BrowserBC into four recurring patterns and give a representative case for each. *(i) Multi-field execution.* Tasks that require setting several interdependent fields in one submission—creating a GitLab issue with both an assignee and a due date, or an admin-panel product with the correct attribute set, price, quantity, size, and category—fail when the agent populates most fields correctly but silently omits one. The skill already enumerates every field; the breakdown is in reliably executing a long form, not in knowing what to enter. *(ii) Ambiguous target selection.* Tasks whose checker requires one *specific* object among several plausible ones—posting a comment that must link a particular repository URL when the user owns many similar repositories—fail when the agent acts on

a near-duplicate. Here the demonstration itself is genuinely under-determined, so the distilled skill cannot disambiguate further. *(iii) Long-horizon budget exhaustion.* Tasks that chain a long information-gathering phase into a final action—reading a dozen product reviews and then composing a forum post—occasionally exhaust the step budget mid-task and terminate on an intermediate page, so the final-state checker sees the wrong URL. *(iv) Generation-limit runaway.* A small number of tasks trigger over-long internal reasoning that hits the model's output ceiling before the action completes; these are properties of the backbone model, not of the skill. Across all four patterns the skill content is correct and applicable—the limit is the agent's execution fidelity, consistent with the boundary identified in §4.2.3. A separate, smaller failure mode is the skill-*induced* regression analyzed above, where an over-specific prior steers an otherwise-capable agent away from the simpler successful path.

4.2.5 Evaluation Integrity

Because skills are natural-language text inserted into the agent context, a naive distillation can leak benchmark-specific answers and inflate results. We take two safeguards. First, distilled skills are constrained to *procedural* content—applicability conditions, navigation, element-selection principles, milestones, and recovery—and we audit the library to remove any checker-targeting content (e.g., copied required substrings, hard-coded field values, or instructions phrased against the grader); the reported numbers are measured *after* this de-leakage pass, so improvements reflect transferable procedure rather than memorized answers. Second, during analysis we identified a small number of genuine *checker* defects in the benchmark itself that penalize correct agent behavior—internally inconsistent task/checker dates, host-alias mismatches between reference URLs and the self-hosted deployment (e.g. `www.reddit.com` vs. the local host), whitespace-sensitive name matching, and stale hard-coded "newest/most-recent" item identifiers that no longer match the deployed data. We correct these defects with normalizations that are additive (they can only turn a previously-missed correct answer into a hit, never the reverse) or anchored to verifiable ground truth, and we apply the identical fixes to *both* conditions. These corrections are independent of the skill library and equally benefit the skill-off baseline. Together, these safeguards ensure the skill-on / skill-off gap measures the value of distilled human procedure, not answer leakage or benchmark artifacts.

4.3 OSWorld Case Study: Skills Beyond Browser-Only Interaction

The WebArena results in §4.2.2 establish the main quantitative result in a browser-agent benchmark. We next use OSWorld-style desktop tasks to ask a different question: whether the same skill abstraction remains meaningful when interaction moves from web pages to a general computer-use environment. We do not treat this as a full OSWorld benchmark. Instead, the section is a diagnostic transfer study: it uses representative desktop cases to identify which part of BrowserBC transfers, which part does not, and what this reveals about the role of natural-language skills in GUI agents.

The key observation is that the transferable object is not a browser-specific action sequence. It is a *procedural prior*: a compact description of preconditions, semantic state transitions, progress milestones, terminal evidence, and recovery strategies. In browser tasks, this prior is expressed over pages, links, forms, and DOM-like structure. In OSWorld-style tasks, the same abstraction is expressed over applications, files, dialogs, window focus, persistent settings, and local artifacts. This shift makes OSWorld useful precisely because it separates procedural knowledge from low-level GUI execution more sharply than browser-only environments.

4.3.1 Case-Study Protocol

We construct 30 OSWorld-style Ubuntu desktop cases spanning system settings, file manipulation, text and structured-data editing, application configuration, terminal interaction, browser/media dispatch, modal dialogs, and office applications. Each case has a fixed initial state and an independent state-based verifier. Within a case, the agent, action interface, budget, and verifier are held fixed; the intervention is whether a distilled semantic skill is available in context.

The skills are distilled from successful GUI demonstrations at the task-family level. They are not coordinate traces or scripts of clicks. A skill records what state must be established, which application-level operation usually produces it, what intermediate evidence indicates progress, and what final evidence should be checked.

Table 2 Mechanistic interpretation of the OSWorld-style case study. Counts summarize selected desktop cases and controls; they are intended to diagnose when skills help rather than estimate a full OSWorld success rate.

Regime	Count	Mechanistic interpretation
High procedural uncertainty, manageable execution	17	No-skill often reaches a relevant surface but lacks a reliable task schema; the skill supplies the missing procedure and terminal evidence.
Low procedural uncertainty	2	The task is shallow enough that the base agent already finds the relevant action path; the marginal value of a skill is small.
High execution fragility	10	The high-level procedure is not the bottleneck; failure comes from focus, modal grounding, file-picker state, browser/session dispatch, terminal robustness, or office-application recovery.
Poor skill calibration	3	A plausible but wrong prior can confidently move the agent away from the target state; skill retrieval and parameter binding are therefore part of the method.

This makes the OSWorld study close in spirit to §4.2.2 while deliberately changing the interaction substrate: from a browser with relatively stable page structure to a desktop with multiple applications, hidden focus state, modal interruptions, and file-system side effects.

In addition to matched skill-on and no-skill runs, we include two forms of controls. First, we keep easy cases where no-skill already succeeds, so the analysis does not attribute all success to skills. Second, we include mismatched-skill controls, where an irrelevant or wrongly parameterized skill is available, to test whether skills behave as useful priors or as brittle instructions that can mislead the agent.

4.3.2 A Mechanistic View of the Outcomes

The case-level outcomes are summarized in Table 2. The counts should be read as diagnostic evidence, not as an OSWorld leaderboard score. The important result is not only that 17 cases improve with matched skills, but that the outcomes organize around three underlying factors: *procedural uncertainty*, *execution fragility*, and *skill calibration*.

This organization gives a stronger interpretation than a pass/fail list. BrowserBC-style skills primarily reduce epistemic uncertainty about the workflow: what to open, what to edit, which state matters, when progress is real, and what evidence terminates the task. They do not directly solve control uncertainty: whether the correct window has focus, whether a dialog button is grounded, whether a file picker opened in the expected directory, or whether an application-specific recovery dialog interrupts the path.

4.3.3 Analysis

Skills as task schemas. The clearest positive cases are those where the no-skill agent partially understands the goal but does not preserve the full task schema. It opens a settings page without toggling the correct state, opens a file without saving the requested edit, searches inside an application without writing the persistent configuration, or opens a terminal without executing the command. These failures are not random; they are failures of task decomposition and completion recognition. The distilled skill repairs this by externalizing a human-like schema: establish the relevant context, perform the state-changing operation, persist it, and verify terminal evidence. This is the same mechanism observed in WebArena, where skills help the agent search, navigate, and recognize completion, but here the schema ranges over desktop applications and files rather than web pages.

A boundary between knowledge and control. The still-hard cases show a complementary pattern. In modal confirmation, archive extraction, clipboard-to-terminal transfer, browser/media opening, and LibreOffice spreadsheet editing, the semantic routine is often plausible or even correct. The failure instead occurs at the level of GUI control: the wrong window receives input, a modal is not grounded, a file path resolves differently than expected, a browser session does not expose the expected title, or an office recovery dialog changes the interaction state. This distinction matters for the paper's claim. Skill distillation is a way to clone procedural knowledge, not a complete solution to perception, focus management, or low-level action reliability. OSWorld makes this limitation visible because desktop tasks contain more hidden state and cross-application transitions than browser tasks.

When are skills unnecessary? Easy cases are not noise; they identify the low-procedural-uncertainty region. If the target control or artifact operation is already exposed and the action path is short, the base agent can succeed without external procedural memory. In such cases, adding a skill may be harmless, but it should not be expected to produce a large gain. This is important experimentally because it prevents the case study from defining success only through examples favorable to BrowserBC.

When can skills mislead? The mismatched controls reveal that a skill is best viewed as a calibrated prior rather than an executable command. A timezone skill that selects London fails for UTC+0 during daylight-saving time, while the matched skill selects Reykjavik. A VLC skill with the wrong parameter writes a plausible but incorrect value. An unrelated sound-slider skill performs confident GUI actions while leaving the VS Code target state unchanged. These cases turn a limitation into a design requirement: BrowserBC-style systems need evidence-aware retrieval and parameterization, so the agent can reject or revise a skill when live state contradicts it.

4.3.4 What Transfers Beyond the Browser?

The OSWorld cases suggest that the portable component of BrowserBC is the representation of procedural skill as a semantic state-transition model. In the browser, the state variables are pages, forms, search results, and submitted mutations. On the desktop, they become windows, files, preferences, dialogs, and local artifacts. The abstraction still works when three conditions hold: the task family has repeated procedural structure, intermediate progress can be recognized semantically, and the final state can be independently verified. Under these conditions, a skill converts exploratory behavior into a reusable workflow.

The abstraction is weaker when any of these conditions breaks. If the task is dominated by fine-grained visual grounding, the skill has little control over the true bottleneck. If the environment state is volatile, a previously valid procedure may land in the wrong window or directory. If the retrieved skill is semantically mismatched, it can add false confidence. Thus, the case study refines the main claim of §4.2.2: skills improve agents by supplying reusable procedural knowledge, but their impact depends on the ratio between procedural uncertainty and execution fragility.

4.3.5 Integrity and Scope

This section does not claim a full OSWorld benchmark improvement. Its purpose is to test external validity of the skill abstraction and to expose the conditions under which it helps, is unnecessary, fails, or misleads. The suite therefore intentionally includes positive cases, easy cases, still-hard cases, and mismatched-skill controls. Skills are constrained to procedural content: they may describe how to open, edit, save, configure, transfer, or verify, but not hidden evaluator targets or checker-specific answers. Within each paired case, the only intended intervention is the availability of the semantic skill.

Overall, the OSWorld case study supports a bounded but meaningful transfer claim. BrowserBC-style skills are not merely browser-navigation tricks. They can encode reusable desktop procedures over applications, files, dialogs, and persistent state. Their value is highest when the missing ingredient is procedural structure; their value is limited when the missing ingredient is low-level GUI grounding or when retrieval supplies the wrong prior.

4.4 9 Website Demo

5 Conclusion

This draft sets up a paper on scalable behaviour cloning for browser agents via distributed skill summarization. The main manuscript should develop the distributed trajectory pipeline, skill extraction procedure, skill consolidation layer, retrieval interface, and evaluation protocol into a complete empirical study. The current template is intentionally clean: old paper content has been removed, while the original formatting system is preserved for the new paper.

[TODO] new cases in appendix

6 Contributions and Acknowledgements

The contributors' names are sorted in alphabetical order of the last name.

Core Contributors

Yuzhao Peng, Houde Qian, Kaisen Yang, Boshi Zhang, Youjie Zheng

References

Léo Boisvert, Megh Thakkar, Maxime Gasse, Massimo Caccia, Thibault Le Sellier De Chezelles, Quentin Cappart, Nicolas Chapados, Alexandre Lacoste, and Alexandre Drouin. Workarena++: Towards compositional planning and reasoning-based common knowledge work tasks, 2025. https://arxiv.org/abs/2407.05291.

Hyungjoo Chae, Sunghwan Kim, Junhee Cho, et al. Web-shepherd: Advancing PRMs for reinforcing web agents. In *Advances in Neural Information Processing Systems*, 2025. https://openreview.net/forum?id=G2kMroO9UV.

Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The browsergym ecosystem for web agent research, 2025. https://arxiv.org/abs/2412.05467.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web, 2023. https://arxiv.org/abs/2306.06070.

Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks?, 2024. https://arxiv.org/abs/2403.07718.

Apurva Gandhi and Graham Neubig. Go-browse: Training web agents with structured exploration. In *International Conference on Learning Representations*, 2026. https://openreview.net/forum?id=IpzRWE52yw.

Hang Gao and Yongfeng Zhang. Memory sharing for large language model based agents, 2024. https://arxiv.org/abs/2404.09982.

Izzeddin Gur, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world webagent with planning, long context understanding, and program synthesis, 2024. https://arxiv.org/abs/2307.12856.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models, 2024a. https://arxiv.org/abs/2401.13919.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Hongming Zhang, Tianqing Fang, Zhenzhong Lan, and Dong Yu. Openwebvoyager: Building multimodal web agents via iterative real-world exploration, feedback and optimization, 2024b. https://arxiv.org/abs/2410.19609.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks, 2024. https://arxiv.org/abs/2401.13649.

Kuan Li, Zhongwang Zhang, Huifeng Yin, et al. Websailor-v2: Bridging the chasm to proprietary agents via synthetic data and scalable reinforcement learning. In *International Conference on Learning Representations*, 2026. https://openreview.net/forum?id=HuP16O5SJf.

Zhaoyang Liu, JingJing Xie, Zichen Ding, et al. Scalecua: Scaling open-source computer use agents with cross-platform data. In *International Conference on Learning Representations*, 2026. https://openreview.net/forum?id=yBFUqdJFZn.

Xing Han Lù, Zdeněk Kasner, and Siva Reddy. Weblinx: Real-world website navigation with multi-turn dialogue, 2024. https://arxiv.org/abs/2402.05930.

H. Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, pages 1273–1282, 2017. https://proceedings.mlr.press/v54/mcmahan17a.html.

Open X-Embodiment Collaboration, Abby O'Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, et al. Open X-Embodiment: Robotic learning datasets and RT-X models. In *2024 IEEE International Conference on Robotics and Automation*, pages 6892–6903, 2024. doi: 10.1109/ICRA57147.2024.10611477. https://arxiv.org/abs/2310.08864.

Viraj Prabhu, Yutong Dai, Matthew Fernandez, et al. WALT: Web agents that learn tools. In *International Conference on Learning Representations*, 2026. https://openreview.net/forum?id=cgIDqcJcoI.

Mohammad Rostami, Soheil Kolouri, Kyungnam Kim, and Eric Eaton. Multi-agent distributed lifelong learning for collective knowledge acquisition. In *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems*, pages 712–720, 2018. https://www.ifaamas.org/Proceedings/aamas2018/pdfs/p712.pdf.

Junhong Shen, Hao Bai, Lunjun Zhang, et al. Thinking vs. doing: Improving agent reasoning by scaling test-time interaction. In *Advances in Neural Information Processing Systems*, 2025. https://openreview.net/forum?id=un1TRwNgiv.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. https://arxiv.org/abs/2303.11366.

Dheeraj Vattikonda, Santhoshi Ravichandran, Emiliano Penaloza, et al. How to train your LLM web agent: A statistical diagnosis. In *Advances in Neural Information Processing Systems*, 2025. https://openreview.net/forum?id=67xkPEM3bZ.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023. https://arxiv.org/abs/2305.16291.

Xinyuan Wang, Bowen Wang, Dunjie Lu, et al. Opencua: Open foundations for computer-use agents. In *Advances in Neural Information Processing Systems*, 2025a. https://openreview.net/forum?id=6iRZvJiC9Q.

Zora Zhiruo Wang, Apurva Gandhi, Graham Neubig, and Daniel Fried. Inducing programmatic skills for agentic tasks, 2025b. https://arxiv.org/abs/2504.06821.

Jialong Wu, Baixuan Li, Runnan Fang, et al. Webdancer: Towards autonomous information seeking agency. In *Advances in Neural Information Processing Systems*, 2025. https://openreview.net/forum?id=quJdphBcdP.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. https://arxiv.org/abs/2404.07972.

Yiheng Xu, Dunjie Lu, Zhennan Shen, Junli Wang, Zekun Wang, Yuchen Mao, Caiming Xiong, and Tao Yu. Agenttrek: Agent trajectory synthesis via guiding replay with web tutorials, 2025. https://arxiv.org/abs/2412.09605.

Qianlan Yang, Xiangjun Wang, Danielle Perszyk, and Yu-Xiong Wang. Self-guided hierarchical exploration for generalist foundation model web agents. In *Advances in Neural Information Processing Systems*, 2025. https://openreview.net/forum?id=9twwDW60Bw.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents, 2023. https://arxiv.org/abs/2207.01206.

Rui Ye, Zhongwang Zhang, Kuan Li, et al. Agentfold: Long-horizon web agents with proactive context folding. In *International Conference on Learning Representations*, 2026. https://openreview.net/forum?id=IuZoTgsUws.

Da Yin, Faeze Brahman, Abhilasha Ravichander, Khyathi Chandu, Kai-Wei Chang, Yejin Choi, and Bill Yuchen Lin. Agent lumos: Unified and modular training for open-source language agents, 2024. https://arxiv.org/abs/2311.05657.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. https://arxiv.org/abs/2308.10144.

Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. Gpt-4v(ision) is a generalist web agent, if grounded, 2024. https://arxiv.org/abs/2401.01614.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024. https://arxiv.org/abs/2307.13854.

A Appendix

A.1 Implementation Details

Add skill-summary prompt format, distributed collection protocol, deduplication/consolidation details, browser runtime details, and retrieval configuration here.

A.2 Additional Results

Add full tables, per-task breakdowns, and qualitative examples here.